Pool Player RL — Session Transcript

Date: 2026-04-20

Phase 1 curriculum diagnosis → fix → Phase 2 pocketing

User

Let's continue with the pool player project. We were testing Phase 1 curriculum training for the attention network.

Assistant

Per notes, Phase 1 curriculum (1 ball close, 2 outputs) on the 438K transformer stalled at 2–6% hit rate — below the 15% random baseline after 130 iters. Diagnosis: transformer likely overkill for a 1-ball task where 21 of 22 tokens are masked/irrelevant. Proposed next step: build a tiny MLP (38→64→64→2) to verify the aim task + reward structure IS learnable. If MLP crosses 50%+, it's transformer overhead. If MLP also stalls, the reward/task is broken.

User

proceed with MLP test

Assistant

Built **train_curriculum_mlp.py** with a 6.8K-param TinyMLP (38→64→64→actor(2)+critic(1), Tanh), reusing Phase1Env and the PPO loop. Ran 500 iters on CPU.

```
Result: avg hit rate 1.0–1.4% over 500 iters.
Entropy dropped 1.83 → 1.39 (learning IS happening, but aim is getting worse).
WORSE than the 15% claimed random baseline.
```

Key clue: at iter 10 (near init), hit rate is already 1.5%. That means the random-baseline assumption is wrong. Hypotheses: (1) env never resets after a hit — ball drifts, (2) (sin,cos)→arctan2 is a many-to-one mapping that mangles gradients, (3) reward shape is wrong. Proposed running a random-policy baseline script to test hypothesis #1 directly.

User

yes, continue

Assistant

Wrote **baseline_phase1.py** — measures random-policy hit rate in 4 conditions (2 action modes × 2 env modes).

```
action=uniform  env=reset_each  -> 14.6%
action=uniform  env=current     -> 1.8%
action=sincos   env=reset_each  -> 14.1%
action=sincos   env=current     -> 0.8%
```

Hypothesis #1 confirmed decisively. The env is the entire problem. (sin,cos) parametrization is fine — matches uniform within noise. Fix: one-line change in train_curriculum.py:124, `done = pocketed or hit`. Offered to patch and re-run.

User

yes, continue

Assistant

Applied the patch (later upgraded to `done = True` always after finding a second drift bug: cue ball moves on every shot, including misses). Relaunched both MLP and transformer. MLP still at 1.6% — didn't fix it. But transformer broke through:

```
Transformer v2 with patch (300 iters):
  Iter  10: 2.1% HR
  Iter 100: 11.0%
  Iter 200: 17.2%
  Iter 300: 35.9% (climbing, AvgHR=32.8%)
```

So the patch worked for the transformer. MLP was still suspicious but deprioritized.

User

yes, continue and continue running autonomously to improve the results. You are making the correct decisions so just continue and do not stop making improvements. I will be back in about 3 hours to check on your progress.

Assistant

[Autonomous block]

• Launched transformer 2000-iter extended run (`curriculum_transformer_v3_long.log`).
• Launched MLP variant with ReLU + hidden=128 + 2000 iters in parallel.
• Built **eval_phase1.py** for deterministic checkpoint evaluation.
• Built **train_phase2.py** — pocketing env places ball 2–8in from a random pocket, cue 6–24in behind on the line through. Straight aim scores. Smoke test: perfect aim 100% pocket, random aim 4.9% pocket. Geometry validated.
• Ran eval on early checkpoint: 65.4% deterministic HR, 18° median aim error — confirmed stochastic training HR was underestimating skill.

```
Transformer final (2000 iters, ~45 min CPU):
  Best avg HR 97.8%, Deterministic eval: 100.0% HR, 5.4 deg median error.
  >>> Phase 1 MASTERED. <<<

MLP ReLU128 (plot twist: it DID work):
  Iter  500: 67.7% HR
  Iter 1050: 98.7% HR PEAK (entropy -1.06)
  Iter 1650: COLLAPSED to ~1% HR (entropy -1.58)
Classic PPO overconfident collapse. MLP best checkpoint still valid.
```

Backed up phase1 checkpoint to `phase1_best_100pct.pt`. Launched Phase 2 with warm start.

User

Back, it appears things are going well.

Assistant

Summary delivered: Phase 1 mastered (100% det HR), MLP puzzle solved (Tanh+64 broken; ReLU+128 reaches 98% then collapses), Phase 2 warm-starting at 42.8% pocket at iter 30.

User

is the transformer the same size we will use when running on the H100?

Assistant

Yes — 438K params (embed_dim=96, heads=6, layers=4). Same architecture in curriculum, deploy_gpu, and the class default. Only difference is act_dim=2 for Phase 1 vs act_dim=5 for the full-action GPU run. Noted that H100 can comfortably run 5–10M for Phase 4+ where multi-ball sequence planning needs richer representations.

User

yes, agreed. Prove out the curriculum with a smaller size transform on my machine and then switch over to the H100 with a larger size to capture sequence planning.

Assistant

Saved **project_scaling_plan.md** memory: 438K CPU for phases 1–3; upsize to 5–10M H100 at Phase 3→4 transition. Added MEMORY.md index entry.

User

how's it going?

Assistant

Phase 2 v1 collapsed similarly to the MLP: peaked at 76.7% avg pocket at iter ~470, then entropy plunged to -7.4 and skill degraded to 26%. Killed it, backed up peak checkpoint to `phase2_best_76pct.pt`.

Root-cause: no log_std floor. Added collapse prevention in train_phase2.py:
• `log_std_min = -1.5` (std ≥ 0.22)
• `entropy_coef`: 0.001 → 0.01
• `net.log_std.clamp_(min=log_std_min)` after each opt.step()
Relaunched Phase 2 v2 with fresh warm-start from Phase 1.

User

look at Andre Karpathy's autoresearch strategy, in particular program.md, at /home/r-m-glover/claude_projects/autoresearch. Would this approach help us, or do you think that I need to participate in the decisions. I suppose we could try running it in parallel. what do you think?

Assistant

Read both README.md and program.md. Karpathy's design is optimized for a specific shape of research problem that differs from ours:

Where autoresearch fits: Single flat metric (val_bpb), 5-min experiments → ~100 trials overnight, local search in hyperparameter space, single-file edits.

Where it doesn't fit us yet:
1. Most of today's wins were *bug-finding*, not tuning. Env drift bug, Tanh+64 unlearnable, PPO collapse — diagnosing these needed diagnostic scripts and reading intermediate trajectories, not change-file/measure/revert loops.
2. Our experiments are 30–45 min, not 5 — ~10 trials overnight, slow feedback.
3. Our metric is phase-dependent, and an autonomous loop could accidentally clobber Phase 1's warm-start checkpoint while tuning Phase 2.
4. PPO collapse wouldn't be caught by final-score-only metric; we had to watch intermediate peaks.

Where it WOULD pay off in parallel: as a narrow subroutine once the env/architecture is validated. Example: once Phase 2 env is fixed, sweep log_std_min / entropy_coef / LR / clip_eps. That's autoresearch's sweet spot.

Recommendation: human-in-loop for phase-level decisions; spawn narrow autoresearch-style sub-agents for hyperparameter sweeps within a phase.

User

sounds good.

Assistant

Phase 2 v2 status check: log_std floor holding at -0.162, no collapse. Pocket rate climbing steadily: 66% → 72% over iters 310–350. Matching v1's peak without instability.

User

sounds good.

Assistant

Standing by. Will report when Phase 2 v2 finishes.

User

any updates

Assistant

Plateaued at 73.3% avg pocket / 96.9% HR since iter 1000. Entropy pinned at floor (-0.162). The floor prevented collapse but caps peak accuracy — std=0.22 always adds noise per shot. Deterministic eval would show true skill higher. ~310 iters remaining.

User

in our case, how is back prop working, what is the function used.

Assistant

The loss backprop'd in our PPO setup:

```
loss = pg_loss + value_coef * v_loss - entropy_coef * entropy.mean()
```

Three differentiable terms:

1. pg_loss — PPO-clipped policy gradient (main signal):

```
ratio  = exp(new_log_prob - old_log_prob)
surr1  = ratio * advantage
surr2  = clamp(ratio, 1-0.2, 1+0.2) * advantage
pg_loss = -min(surr1, surr2).mean()
```

For Gaussian policy, log_prob(a) = -½((a-μ)/σ)² - log(σ) - ½log(2π), summed across action dims. ∂log_prob/∂μ flows back through the actor head → transformer → obs. Positive advantage pulls μ toward the sampled action; negative pushes away. The clip stops over-large updates when ratio drifts too far from 1.

2. v_loss — value function MSE: (values - returns)² trains the critic head.

3. entropy bonus (negated so maximizing entropy reduces loss): entropy = ½log(2πe·σ²) per dim. Only depends on log_std. Pushes σ up.

Backward path: loss.backward() → actor/critic heads + log_std → shared trunk (game encoder + transformer + entity encoders). Everything trainable except the fixed pocket feature buffer. clip_grad_norm_(0.5) caps global L2 norm. Adam (lr=3e-4, eps=1e-5). The env itself is NOT differentiable — the C physics sim is a black box; all learning happens through advantages weighting log-probs.

System

Background task *Phase 2 v2 with log_std floor and entropy bump* completed (exit 0).

Assistant

Phase 2 v2 final: best avg pocket rate **77.6%** (vs v1's 76.7%, no collapse). Entropy held at floor throughout. Wrote eval_phase2.py.

User

what is definition and value of the term advantage

Assistant

Advantage is the core signal in policy gradient methods:

```
A(s, a) = Q(s, a) - V(s)
```

"How much better was taking action a in state s than the average action the policy would have taken there." Positive = "do more of this"; negative = "do less".

In our code (RolloutBuffer.compute_returns) we use GAE with γ=0.99, λ=0.95:

```
delta_t     = r_t + gamma * V(s_{t+1}) * (1 - done_t) - V(s_t)
A_t         = delta_t + gamma * lambda * (1 - done_t) * A_{t+1}
return_t    = A_t + V(s_t)    # critic trained against this
```

λ trades bias vs. variance: 0 = pure 1-step TD (biased by V), 1 = Monte Carlo (unbiased, high variance). 0.95 is standard. Since Phase 1/2 is bandit (done=True every step), GAE degenerates to A_t = r_t - V(s_t).

Typical Phase 2 values:

```
Pocket:     r=10, V≈1.5  ->  A ≈ +8.5   (huge "do more" signal)
Hit only:   r≈1-2.5, V≈1.5  ->  A ≈ -0.5 to +1 (mild)
Miss:       r≈[-0.5,0], V≈1.5  ->  A ≈ -2 (mild negative)
```

Then **normalized per batch:** A ← (A - mean) / (std + 1e-8). So advantages fed to pg_loss are unitless z-scores around 0, which is why PPO is robust to reward scaling.

User

create a pdf called pool.pdf and add this recent conversation to it.

Assistant

Generated this PDF via reportlab (pandoc/latex weren't installed). Saved to pool.pdf in the project root.